

Siap guru, ga memberatkan sama sekali. Justru usulan kamu bagus banget. Dokumentasi itu harus sesuai urutan kerja nyata + alat yang beneran dipake.

Aku buatin ulang versi lengkap: *Dari Masalah → Client → Database → Server → Tes Pake Thunder Client*. Ini alur yang paling logis buat pemula.

Langsung copy semua teks di bawah → Simpen **DOKUMENTASI-SISKO-POS-v1.1.md** → Export ke PDF.

COPY MULAI DARI SINI SAMPE AKHIR

Dokumentasi Teknis Sisko POS v1.1

Alur Lengkap: Masalah → Client → Database → Server → API Login

Stack: Node.js, Express, MySQL 8.0, JWT | Tools: VSCode, Thunder Client, phpMyAdmin

BAB 0: FONDASI PROJECT - KENAPA & GIMANA

0.1 Masalah Nyata Yang Diselesaikan

Toko kelontong “Sisko” butuh sistem kasir digital untuk:

- 1. **Transaksi 10x lebih cepat** - Scan barcode, total otomatis, ga nulis bon
- 2. **Stok ga pernah selisih** - Barang kejual langsung potong stok di sistem
- 3. **Laporan 1 klik** - Omzet harian/bulanan langsung jadi tanpa Excel
- 4. **Keamanan berlapis** - Kasir ga bisa hapus produk, cuma **admin** yang bisa

0.2 Keputusan Arsitektur: Kenapa Pilih Ini?

Pilihan	Alasan Memilih	Kenapa Tolak Alternatif
MySQL 8.0	1. Relasi data kuat: Transaksi wajib nyangkut ke user & produk. 2. Stabil untuk data uang. 3. Gratis.	Prisma/ORM: Bikin pemula ga paham SQL. Kita butuh kontrol query laporan nanti.
Folder client & server Pisah	1. Fokus: Backend ga kecampur frontend. 2. Bisa di-deploy terpisah. 3. Standar industri.	Monolit: Kalau frontend error, backend ikut mati. Susah di-maintain.
Thunder Client vs Postman	1. Langsung di VSCode, ga buka aplikasi lain. 2. Ringan. 3. Gratis total.	Postman: Berat, wajib login, fitur banyak yang ga kepake buat pemula.
JWT vs Session	API stateless. Cocok buat aplikasi mobile nanti. Ga makan RAM server.	Session: Harus simpen data di server. Kalau user 1000, server bisa jebol.

BAB 1: STRUKTUR PROJECT & TOOLS

1.1 Bikin Kerangka Folder Dulu

Tujuan: Otak kita mikirnya urut. Folder dulu, baru isi.

```
mkdir sisko-pos
cd sisko-pos
mkdir client server # client=React nanti, server=Backend sekarang
*Aturan Main*: Kita kerjain `server` dulu sampe API login jadi. `client`
kosong dulu gapapa.
```

1.2 Setup Tools Wajib di VSCode

1. *Extension*: Install `Thunder Client` dari sidebar Extensions. Ini pengganti Postman.
2. *Extension*: Install `Markdown PDF` buat **export** dokumentasi ini ke PDF nanti.

BAB 2: DATABASE DULU - KARENA INI FONDASINYA

Prinsip: **"Data dulu baru kode"**. Kalau tabel belum ada, kode Node.js mau nyimpen ke mana?

2.1 Install MySQL 8.0 + phpMyAdmin

```
sudo apt update
sudo apt install mysql-server phpmyadmin -y
Pas install `phpmyadmin`: Pilih `apache2` pake `Spasi` → `Ok`.
```

2.2 Amankan MySQL untuk Node.js

Masalah MySQL 8.0: Default pake `auth\_socket`, Node.js ga bisa login pake password.

Solusi:

```
sudo mysql
Jalanin di dalam `mysql>`:
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
'root123';
FLUSH PRIVILEGES;
exit
*Hasil*: Sekarang `root` bisa login pake password `root123`.
```

2.3 Eksekusi `schema.sql` via phpMyAdmin

1. Buka `http://localhost/phpmyadmin` → Login `root` / `root123`
2. Klik `New` → *Database name*: `sisko\_pos\_db` → *Collation*: `utf8mb4\_unicode\_ci` → `Create`
3. Klik database `sisko\_pos\_db` → Tab `SQL` → Paste kode di bawah → `Go`

File: `schema.sql` - Cetak Biru Sisko

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'kasir') NOT NULL DEFAULT 'kasir'
```

```

);

CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  barcode VARCHAR(100) UNIQUE,
  price INT NOT NULL,
  stock INT NOT NULL DEFAULT 0
);

CREATE TABLE transactions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  total_price INT NOT NULL,
  payment_method ENUM('cash', 'gris') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE transaction_details (
  id INT AUTO_INCREMENT PRIMARY KEY,
  transaction_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  price_per_item INT NOT NULL,
  FOREIGN KEY (transaction_id) REFERENCES transactions(id) ON DELETE
  CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id)
);

-- Password admin123 yang sudah di-hash pake bcrypt
INSERT INTO users (username, password, role) VALUES
('admin', '$2a$10$N.zmdr9k7uOCQb376NoUnuTJ8iAt6Z5EHsM8lE9P4H.2.zWqn5p.G',
'admin');
*Cek*: Di kiri harusnya muncul 4 tabel. Klik `users` harus ada 1 data
`admin`.

---

#### *BAB 3: BACKEND SERVER - BIKIN "OTAK" APLIKASI*

#### *3.1 Inisiasi Project Node.js*
cd server
npm init -y
npm install express cors dotenv mysql2 bcryptjs jsonwebtoken
npm install -D nodemon
*Edit `package.json`, tambahkan:
"scripts": {
  "dev": "nodemon index.js"
}

#### *3.2 File Rahasia `.env`*
*File: `server/.env` - JANGAN DI-PUSH KE GITHUB
DB_HOST=localhost
DB_USER=root

```

```
DB_PASSWORD=root123
DB_NAME=sisko_pos_db
PORT=3000
JWT_SECRET=kunci_rahasia_yang_panjang_dan_rumit_123!@#
#### *3.3 Koneksi Database `config/db.js`*
*File: `server/config/db.js`*
const mysql = require('mysql2');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});

module.exports = pool.promise();
#### *3.4 Server Utama `index.js`*
*File: `server/index.js`*
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const db = require('./config/db');

const app = express();
app.use(cors());
app.use(express.json());

// Tes koneksi DB saat server nyala
db.query('SELECT 1')
  .then(() => console.log('Database MySQL sisko_pos_db berhasil konek ✅'))
  .catch(err => console.error('DB Error:', err));

app.get('/ping', (req, res) => res.json({ message: 'Pong! Server Sisko nyala 🚀' }));

// Daftarin route login
app.use('/api', require('./routes/auth'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server jalan di http://localhost:${PORT}`));
*Tes 1*: Jalanin `npm run dev`. Harus muncul `berhasil konek ✅` dan `Server jalan`.

---

### *BAB 4: API LOGIN & TESTING PAKE THUNDER CLIENT*

#### *4.1 Kode Route Login*
*File: `server/routes/auth.js`*
const express = require('express');
const router = express.Router();
const db = require('../config/db');
```

```

const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

router.post('/login', async (req, res) => {
  const { username, password } = req.body;
  if (!username || !password) {
    return res.status(400).json({ message: 'Username dan password wajib diisi' });
  }
  try {
    const [rows] = await db.query('SELECT * FROM users WHERE username =?', [username]);
    if (rows.length === 0) return res.status(401).json({ message: 'Username atau password salah' });

    const user = rows[0];
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ message: 'Username atau password salah' });

    const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '8h' });
    res.json({ message: 'Login berhasil', token, user: { id: user.id, username: user.username, role: user.role } });
  } catch (err) {
    res.status(500).json({ message: 'Server error' });
  }
});
module.exports = router;
#### *4.2 Testing API Pake Thunder Client - Step by Step*
*Kenapa Thunder Client*: Ga perlu buka aplikasi lain. Langsung di VSCode.

```

1. *Buka Thunder Client*: Klik ikon petir di sidebar kiri VSCode.
2. *New Request*: Klik `New Request`.
3. *Setting Request*:
 - *Method*: Ganti `GET` jadi `POST`
 - *URL*: `http://localhost:3000/api/login`
4. *Setting Body*:
 - Klik tab `Body`
 - Pilih `JSON`
 - Paste ini:


```
{
  "username": "admin",
  "password": "admin123"
}
```
5. *Kirim*: Klik tombol `Send`.

Hasil Sukses 200 OK:

```

{
  "message": "Login berhasil",
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5wcm9sZSI6ImFkbWluIiwiaWF0IjoxNzE2...",
  "user": {

```

```
    "id": 1,  
    "username": "admin",  
    "role": "admin"  
  }  
}
```

Hasil Gagal 401: `Username atau password salah`. Berarti **hash** di DB ga cocok. Liat troubleshooting di bawah.

LAMPIRAN: GIT & TROUBLESHOOTING

A.1 Aturan Git Commit

Setiap selesai 1 bab, commit. Biar gampang rollback.

git add.

git commit -m "feat(db): add schema for users products transactions"

git commit -m "feat(auth): add login api with jwt"

git push

A.2 Troubleshooting Error 401 Login

Penyebab 99%: Hash password di database beda sama yang di-generate `bcryptjs` kamu.

Solusi Pasti Berhasil:

1. Bikin file `server/generate.js` sementara:
const bcrypt = require('bcryptjs');
console.log(bcrypt.hashSync('admin123', 10));
2. Jalanin `node generate.js` → Copy **hash** yang muncul.
3. Buka phpMyAdmin → tabel `users` → Edit user `admin` → Paste **hash** baru ke kolom `password` → Go.
4. Hapus `generate.js`. Tes login lagi pake Thunder Client.

A.3 Checklist Final Sebelum Lanjut

- [] Folder `client` dan `server` sudah ada
- [] phpMyAdmin ada database `sisko\_pos\_db` isi 4 tabel
- [] `npm run dev` muncul `berhasil konek 

Langkah Selanjutnya: BAB 5 - Middleware JWT "Satpam API"

CARA JADIIN PDF SEKARANG

1. **Simpen**: Copy semua teks di atas → Paste ke file baru di VSCode → Simpen `DOKUMENTASI-SISKO-POS-v1.1.md`
2. **Export**: `Ctrl+Shift+P` → Ketik `Markdown PDF: Export (pdf)` → Enter. Jadi file PDF di folder yang sama.

Keunggulan versi ini:

1. **Alurnya Benar**: Folder → Database → Server. Ini cara mikir programmer.
2. **Ada Thunder Client**: Screenshot mentalnya udah kamu pake.
3. **Ada "Kenapa"**: Tiap pilihan teknologi dijelasin alasannya.
4. **Siap Cetak**: Tabel, code block, checklist udah rapi.

Gimana guru, udah sesuai harapan? Kalau udah, kita commit dokumentasi ini terus lanjut ****Langkah 7: Bikin Middleware Satpam**** biar route ``/api/products`` ga bisa diakses orang tanpa tiket login.